DHCP, NAT, and IPv6

Lecture 23

Computer Networks and Distributed Programming

The address nobody typed in

- We can now find out where a network is broken. What we never asked is where the address of *this* machine came from.
- Nobody typed it in. It was asked for, granted for a limited time, and it can be a different one tomorrow.
- The other question is the one the traces kept running into: a whole network living behind a single public address.
- Today: the protocol that hands addresses out, the translation that makes one address serve a house, and the successor to IPv4.
- The three belong together. All of them exist because thirty-two bits were not enough, and have not been for thirty years.

Two ways to fill in an address

- A block of addresses is not a configuration. Somebody still has to give one address to each interface.
- By hand: the administrator writes address, mask and gateway into the machine. This is how router interfaces are done.
- Automatically: the network gives a free address to whatever arrives. This is how host interfaces are done.
- Doing hosts by hand means keeping a list of which addresses are free and never being wrong about it.
- And it does not survive movement: a laptop carried from a room to a library to a lecture hall needs a new address in each place.
- The automatic way is a protocol, and it is the one running on every machine in this room.

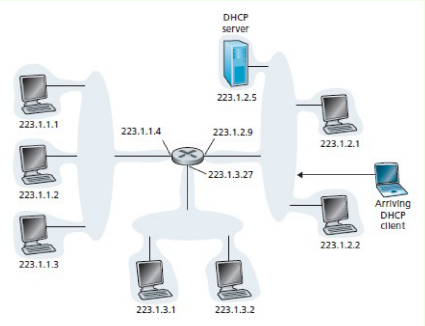
What a host needs before it can send anything

An address alone gets a machine nowhere. Four things arrive together.

- An address: and a mask: which destinations are on my own subnet, and which ones are not.
- A first-hop router: where everything that is not on my subnet has to go — the default gateway.
- A local DNS server: without which every name the user types is unresolvable.
- DHCP carries all four in one exchange, which is why it is called a plug-and-play, or zeroconf, protocol.
- The alternative is those four values typed into every machine by somebody who knows them. We will look at that window later today.

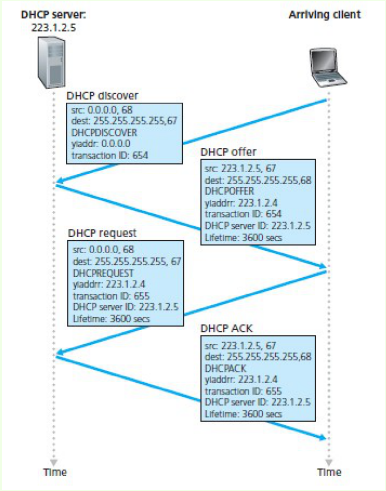
The server is often the router itself

- DHCP: a client-server protocol. The client is a host that has just arrived; the server is whatever answers on that subnet.
- Here it is a machine on `223.1.2.0/24`. In a house it is almost always the router, which does both jobs.
- It runs over UDP, port 67 at the server and 68 at the client: formally an application-layer protocol.
- Which is worth a moment. The thing that configures the network layer sits at the top of the stack and uses it like any other application.



Four messages

- Discover: the arriving host asks whether any server is listening. It knows nothing yet, not even where it is.
- Offer: a server proposes an address in `yiaddr` — *your* address — with a mask and a lease time.
- Request: the client picks one offer and echoes it back, naming the server whose offer it took.
- ACK: the server confirms. Only now is the address the client's, and only until the lease runs out.
- A transaction ID ties the four together: none of them is addressed to anybody in particular.



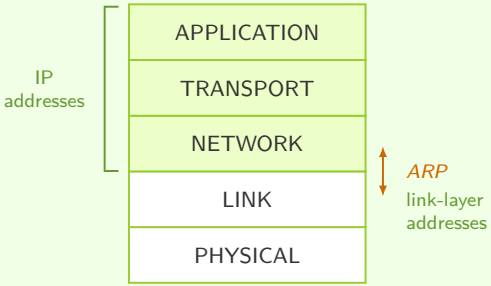
Addressing a host that has no address

The client has no address. What goes in the source field, and how can an answer ever come back to it?

Source 0.0.0.0, *this host*; destination 255.255.255.255, *everybody here*. The reply is broadcast too: neither side can address the other yet, so both shout.

- What actually delivers those two datagrams is the layer below, which has addresses of its own and a broadcast of its own.
- Every interface carries one, fixed when it was manufactured. It is not an IP address and it was never configured by anyone.
- A subnet broadcast is not forwarded by routers, so the whole exchange is confined to one subnet by construction.
- Which is why the four messages only work when the server is on the same subnet as the client.

Which address belongs to which layer



- IP addresses: are what the network layer and everything above it use to name an interface. Prefixes, masks, subnets: all up there.
- The link layer: names interfaces its own way, knows nothing of subnets, and only ever delivers across one hop.
- Between the two sits ARP: given an IP address on my own subnet, it produces the link-layer address to put on the frame. We take it apart at the link layer.

When the server is on another subnet

- One server per subnet is wasteful, and a discover message cannot leave the subnet it was shouted into.
- Relay agent: a router configured to pick that message up and forward it, as an ordinary unicast datagram, to a server elsewhere.
- It adds the address of the interface the request came in on, so the server knows which pool the address must come from.
- In the network we just looked at, one server on the right can then serve all three subnets, and the router does the carrying.
- Several servers may also answer the same discover. The client takes one offer and names it in the request; the others are released.

Reading what the lease gave you

```
$ ip addr show wlp9s0
2: wlp9s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    link/ether 34:e6:ad:f0:d5:11          the address of the layer below
    inet 100.103.0.64/16 brd 100.103.255.255 scope global dynamic wlp9s0
       valid_lft 84532sec preferred_lft 84532sec
    inet6 fe80::47c4:ec38:2dc3:5d70/64 scope link
$ ip route
default via 100.103.0.1 dev wlp9s0 proto dhcp metric 600
100.103.0.0/16 dev wlp9s0 proto kernel scope link src 100.103.0.64
```

- `dynamic` and `valid_lft` say it outright: this address is on loan, and the countdown is running.
- The default route was not typed in either — `proto dhcp` names who put it there.
- Three of the four things a host needs are on this screen. The fourth, the resolver, went into a file.

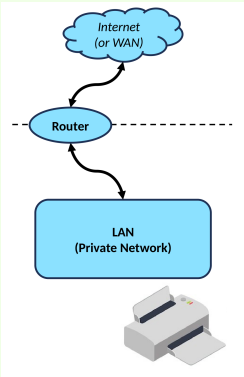
A lease is temporary, and so is the connection

- A lease is hours or days. A client that wants to keep its address renews it before the time runs out.
- If it does not, the address goes back into the pool and is offered to somebody else. Nothing was promised beyond the lease.
- Now move the machine: to another subnet and it is given a different address — which is the only correct thing to do there.
- But a connection is named by four values, and one of them has just changed. Every connection open at that moment is dead.
- Nothing at this layer repairs that. Keeping one address while changing network is separate machinery, built for cellular networks.

Does everything have to be reachable

Should your printer be usable by anybody on the Internet?

- The rule so far: every interface carries an address unique in the entire Internet.
- Addresses are finite: a house with fifteen devices is not given fifteen public addresses.
- Exposure: most of those devices have no business being addressable from outside at all.
- Nobody knows the block: a homeowner cannot be expected to know which addresses of the provider's block are free.

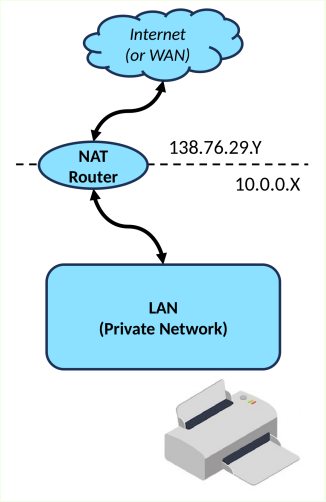


One address for the whole house

Network address translation

A router rewrites the addresses and ports of the datagrams crossing it, so that an entire network appears from outside as one address.

- Behind it is a private network, a realm with private addresses: they mean something only inside.
- The same addresses are in use in a million other houses right now, and no two of them will ever meet.
- From outside the box is not a router at all: it is one machine with one address. Hence the other name, masquerading.

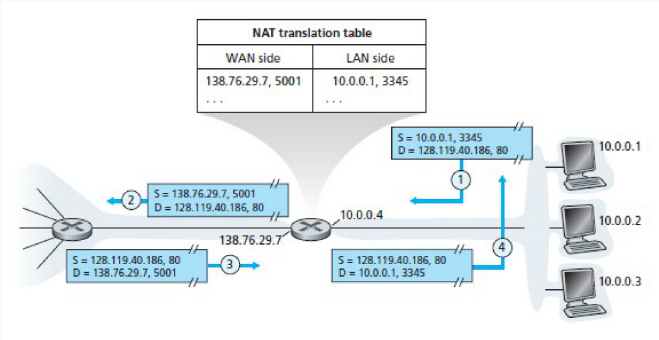


The addresses that were never handed out

Block	Range	Meaning
10.0.0.0/8	10.0.0.0 – 10.255.255.255	private
172.16.0.0/12	172.16.0.0 – 172.31.255.255	private
192.168.0.0/16	192.168.0.0 – 192.168.255.255	private
127.0.0.0/8	127.0.0.1 in practice	this machine

- No registry ever assigns these. They are held back precisely so that every private network can use them at the same time.
- A router in the public Internet drops a datagram addressed to one of them: there is no such place out there to send it to.
- We have met the others already — the lowest and the highest address of every block, and 255.255.255.255.

The translation table



- Going out: source address and port are replaced by its own address and a fresh port, and the pair goes into the table.
- Coming back: the destination is looked up in that same row and the private address and port are put back.
- The web server never finds out: it answers the address and port it was asked from.

Why the port cannot stay the same

Two machines in the house both pick source port 3345. What does the router write in the table?

Not 3345, twice. Once the source address is the same for both, the port is the only thing telling the two conversations apart, so the router assigns a new one to each and remembers which is which.

- Neither host knows anything about the other's traffic, so nothing prevents the collision. It is ordinary, not exotic.
- A port number is 16 bits, so one public address carries over 60,000 conversations at once.
- Which is also the ceiling: what runs out at that boundary is the port space, not the address.
- And notice what the router is doing to get there — reading and rewriting a transport-layer field.

Nobody can start from outside

A host on the Internet sends the first datagram to `10.0.0.1`. What does the router do with it?

Nothing useful. That row of the table exists only because the conversation was started from inside. With no row, there is nothing to say which of the machines behind the router the datagram is for.

- Port forwarding: a row written by hand: whatever arrives on this port goes to that machine. It is the answer for one service.
- The general answer is a family of techniques called NAT traversal, where a third machine outside helps two insiders find each other.
- Every peer-to-peer application has to deal with this. So does the server you wrote: on your own LAN it answers, from outside it does not exist.

Your own address is not yours either

Is `100.103.0.64`, from the configuration we read a moment ago, one of the private addresses?

It is in none of the three blocks — and it is not a public address of yours either. It belongs to 100.64.0.0/10, *shared address space*, which providers use to number their own customers behind a translator of their own.

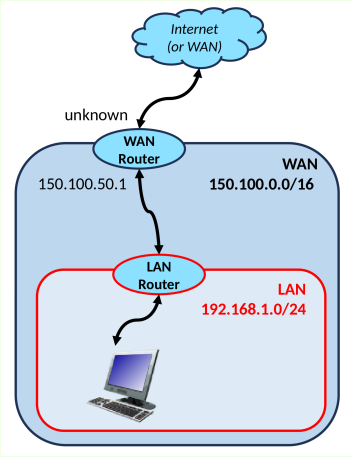
- Two translations in a row, then: one in the house, one at the provider. Nothing between that machine and the Internet is its own.
- The first hop of a trace from it has an address from the same block: a router of the provider, not a router of the Internet.
- Consequence: configuring the home router is not enough to be reachable from outside, because the outer translator is not yours.
- This is what running out of addresses looks like from a laptop, and it is the argument for the rest of the lecture.

What translation costs

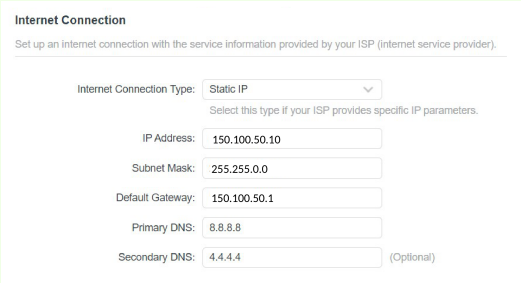
- Ports address processes: not hosts. Here the same field is used to tell hosts apart, which is a different job done with the same bits.
- Routers are network-layer devices: in principle. This one reads and rewrites transport headers.
- And it holds state: lose the table and every conversation through it dies. A router that only forwards has nothing to lose.
- The end-to-end argument says two hosts should be able to talk without anything in between altering what they said. This alters it.
- And it worked well enough to postpone the real answer by twenty-five years. We get to that answer at the end of the hour.

A LAN inside somebody else's network

- The administrator of the surrounding network gives us three numbers and nothing else.
- The network: `150.100.0.0/16`, of which exactly one free address is ours: `150.100.50.10`.
- The way out: `150.100.50.1`, the gateway of that network.
- Everything on the inside is our decision. We take 192.168.1.0/24 for the new LAN, out of the reserved space.
- One address on the outside, 254 usable ones on the inside, and a translating router between them.



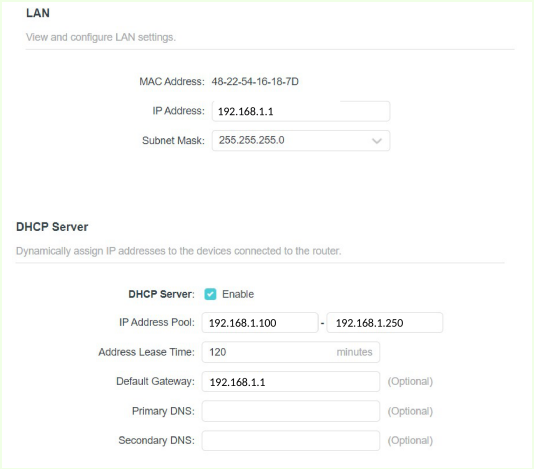
The router's outside face



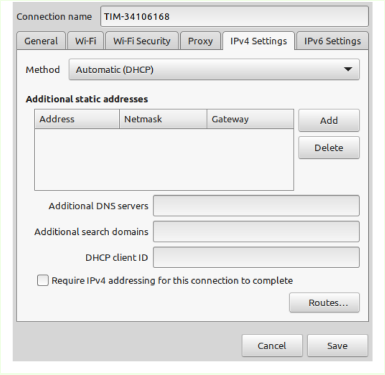
- Seen from the network it is joining, the router is just another host: one interface, one address.
- The four fields are the four things any host needs — address, mask, gateway, resolver.
- Static: because the administrator handed us a fixed address — had that network run a server, this box would be a client on one side and a server on the other.

The router's inside face, and its DHCP server

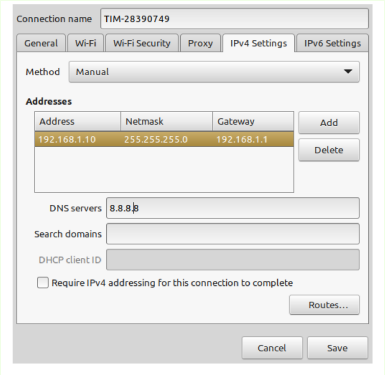
- The other interface, in our own subnet: `192.168.1.1`, since the router conventionally takes `.1`.
- Pool: the range the server may give away, `.100` to `.250`. What is outside it we assign ourselves.
- Lease time: 120 minutes here. Short leases recycle addresses quickly, long ones survive a machine being switched off.
- The gateway and resolver fields are what this server will tell its clients — and the gateway it announces is itself.



Ask, or be told



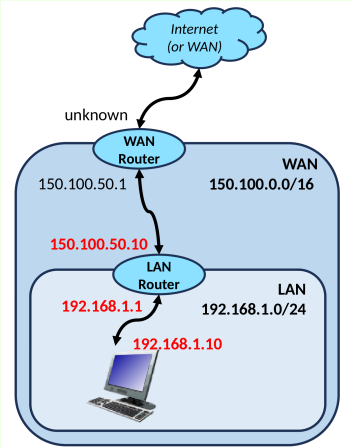
Automatic — one field is set and the rest are empty: the server fills them in, and this machine knew nothing



Manual — the same four values, typed in by somebody who knew them. Afterwards nothing tells the two apart

Inside the pool, outside the pool

- The finished network: two addresses on our router, one in each subnet, and a host configured by hand at `192.168.1.10`.
- `.10` is below the pool, and that is deliberate.
- If it were not: the server would one day offer `.10` to an arriving machine, and two interfaces on one subnet would answer to the same address.
- Everything the host sends goes to `192.168.1.1`, which translates it and forwards it to `150.100.50.1`. The host knows about neither.



Running out

- By the early 1990s the address space was visibly finite, and the IETF began work on a successor.
- Two estimates from the people who were measuring it: the addresses would be gone in 2008, or in 2018.
- February 2011: the last unallocated block left the central pool for a regional registry. The registries have been rationing since.
- Translation is a large part of why the deadline kept moving. It did not create addresses; it made each one serve more machines.
- And IPv5: the number was spent on an experimental streaming protocol, ST-2, which was dropped. The successor got 6.

What 128 bits buy

- 128-bit addresses: 3.4×10^{38} of them. Nobody will ever again design a scheme to save addresses.
- A fixed 40-byte header: nothing to measure and nothing optional in the way: a router finds every field by position.
- Flow label: 20 bits naming a stream of datagrams that want the same treatment — a video, say, rather than a mail transfer.
- Anycast: a new kind of address altogether: deliver this to any one of a group. The nearest mirror, the nearest resolver.
- What a flow is exactly was left open in the specification and has stayed open. The field is there for when it is settled.

Writing an IPv6 address down

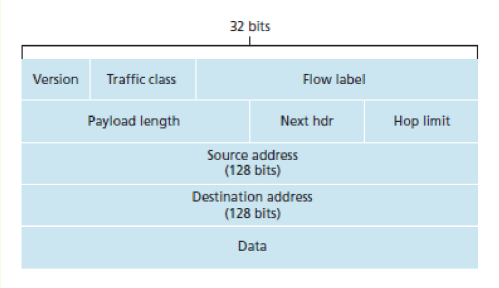
`8000:0000:0000:0000:0123:4567:89AB:CDEF`

is written

`8000::123:4567:89AB:CDEF`

- Eight groups of four hexadecimal digits, separated by colons. Sixteen bytes in dotted decimal would be unreadable.
- Leading zeros: inside a group may be dropped, so `0123` becomes `123`.
- A run of zero groups: may be replaced by `::`, and only once per address: twice and the length is ambiguous.
- An IPv4 address may be written after a pair of colons, as in `::192.31.20.46`.
- The `fe80::` line in the configuration we read earlier is one of these. That machine has had an IPv6 address all along.

The header in one look



- First word: version, traffic class, flow label: which protocol, what priority, which stream.
- Second word: payload length, next header, hop limit: how much follows, what it is, how far it may still go.
- Then the two addresses, sixteen bytes each: thirty-two of the forty bytes are addresses, and no field measures a fixed-size header.

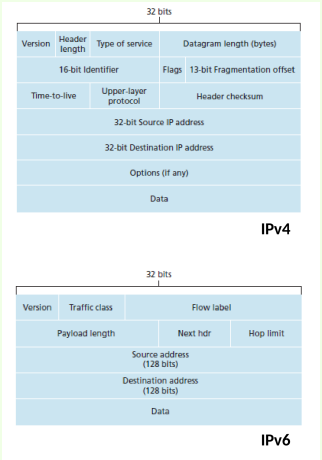
Same job, new name

IPv4	IPv6	
time to live	hop limit	the same counter, honestly named
protocol	next header	the same values, and one more job
type of service	traffic class	the same job
total length	payload length	no longer counts the header

- The renaming is not cosmetic. Time to live was specified in seconds and used as a hop count by everybody; the new name says what it is.
- Next header: does two jobs: it names the transport protocol, or it points at the first of a chain of optional headers.
- Payload length: counts what follows the header, not the whole datagram — so a payload can now be a full 65,535 bytes.

What was thrown away

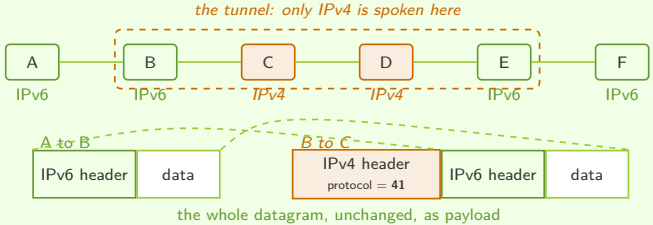
- Fragmentation: routers no longer cut datagrams up. One too large is dropped and the sender is told *packet too big* — the message we read when diagnosing paths.
- Header checksum: it had to be recomputed at every hop, because the hop count changes at every hop. Transport and link layers were already checking.
- Options: out of the fixed header and into the chain of next headers, where a router that does not care skips them.
- Everything removed was removed for one reason: it was work done at every hop, by every router, on every datagram.



There is no flag day

- The problem: a new machine can be built to speak both, but the IPv4 machines already deployed cannot be taught IPv6. There is no backward compatibility.
- A flag day: a date on which everything is switched off and upgraded. It was done once, for the move to TCP, on a network of a few hundred machines, and it was painful even then.
- With billions of devices and nobody in charge of all of them, the idea is not discussable.
- What is done instead: the two protocols coexist. Most machines today run both at once and choose per destination — which is why our capture had an address of each kind.
- Where a stretch of the path speaks only IPv4, the datagrams have to be carried across it somehow.

A tunnel is an encapsulation



- Two IPv6 routers with only IPv4 between them: the entire IPv6 datagram becomes the payload of an IPv4 datagram addressed to the far end.
- The routers in between forward an ordinary IPv4 datagram and never learn what they are carrying.
- Protocol 41: the number in the IPv4 header that tells the far end what is inside, exactly as 6 says TCP and 17 says UDP.
- The far end unwraps it and forwards the original. Nothing is translated: what comes out is what went in.

Why this layer is the hard one to change

- Adoption was slow for a decade and then began to move: in 2020: about a quarter of the clients reaching Google did so over IPv6.
- Mobile networks pushed hardest. An operator numbering millions of handsets has no other option left.
- The general lesson: the network layer is the one thing every machine in the Internet has to agree on, and the agreement is the expensive part.
- The application layer changes constantly — HTTP was rewritten twice while this was going on — because only two endpoints have to agree each time.
- Twenty-five years, for a protocol whose only serious change is the width of an address.

Summary

- Addresses inside a block are handed to hosts by DHCP: discover, offer, request, ACK — broadcast, and confined to one subnet unless a router relays them.
- What arrives is an address, a mask, a gateway and a resolver, and it arrives on a lease. Move to another subnet and the open connections die with the old address.
- NAT lets one public address serve a private network by rewriting addresses and ports and remembering the pairs.
- It only works for conversations started from inside, it puts state in the middle of the network, and it is what made IPv4 last this long.
- IPv6 is 128-bit addresses and a fixed 40-byte header: no fragmentation at routers, no checksum, options in a chain, and a transition with no end date.
- What we still have not asked is where the rows of a forwarding table come from. That is the next question.